
task-app 30日間カリキュラム

Day 21

統計カードを表示しよう

Next.js 15 / TypeScript / Prisma / tRPC

目次

前回の振り返り	3
今日のゴール	3
なぜこれを作るのか	4
実装ステップ一覧	6
Step 0: レポート集計 API(getOverview)を自分で書く(読む目安: 14分).....	8
Step 1: サーバー集計の考え方(読む目安: 3分)	22
Step 2: import 文を書く(読む目安: 3分).....	24
Step 3: ページの骨組みを作る(読む目安: 5分)	28
Step 4: データを取得する(読む目安: 3分).....	32
Step 5: 受け取った概要データを読む(読む目安: 5分).....	33
Step 6: ローディング判定を追加(読む目安: 3分)	35
Step 7: 統計カードを表示する(読む目安: 5分).....	37
Step 8: プロジェクト統計テーブル(読む目安: 5分)	42
Step 9: 動作確認(読む目安: 3分).....	48
完成コード全体	53
今日のまとめ	83
つまずきポイント	83
今日学んだ用語	84
理解チェック	84
追加課題: 未対応の件数をカードにする.....	85
次回予告	86
次に読むもの	86
奥付	87

前回の振り返り

Day 20 ではキーワードや複数フィルターでタスクを検索するページを作りました。
今日は集計データを統計カードで表示します。

今日のゴール

レポートページに統計カードを表示します。完成版と同じく

`api.report.getOverview` からサーバーで集計済みのデータを受け取り、総タスク数・完了率・合計作業時間・平均作業時間を 4 枚のカードに分けて表示します。

完成イメージ: 4 枚の統計カードとプロジェクト統計テーブルが並んだレポートページです。

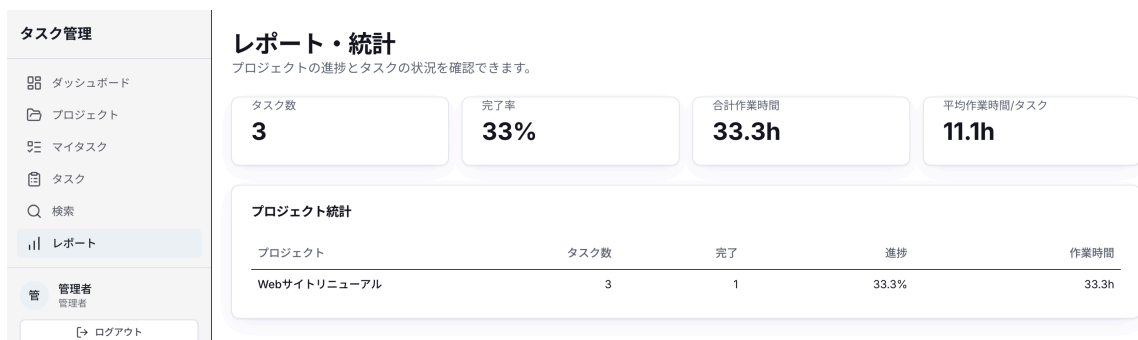


図1: レポートページ。タスク数・完了率・合計作業時間・平均作業時間の4枚のカードが横に並び、その下にプロジェクト統計のテーブルが出ている

数字は初期データのものです。読者の手元と件数が違ってても実装の誤りではありません。

なぜこれを作るのか

タスクが増えるほどいま全体がどれくらい進んでいるかは一覧をスクロールするだけでは把握しにくくなります。完了率や作業時間を集計して数字で見せることで、進み具合をひと目で確認できるようにします。

例えば: タスクを10個登録したとき「いくつ終わったっけ?」と1つずつ数えるのは面倒です。統計カードがあれば完了率も作業時間もまとめて目に入ります。

今日のスコープ

区分	内容
対象ファイル	<code>src/app/report/page.tsx</code> 、 <code>src/server/api/routers/report.ts</code> 、 <code>src/server/api/root.ts</code> 、 <code>src/component/layout/app-layout.tsx</code>
今日作る範囲	統計カード4枚 + プロジェクト統計テーブル
実コードとの違い	実コードにはグラフ(Day 22)や週次リンク(Day 23)もありますが、今日は扱いません

レポートページのデータフロー

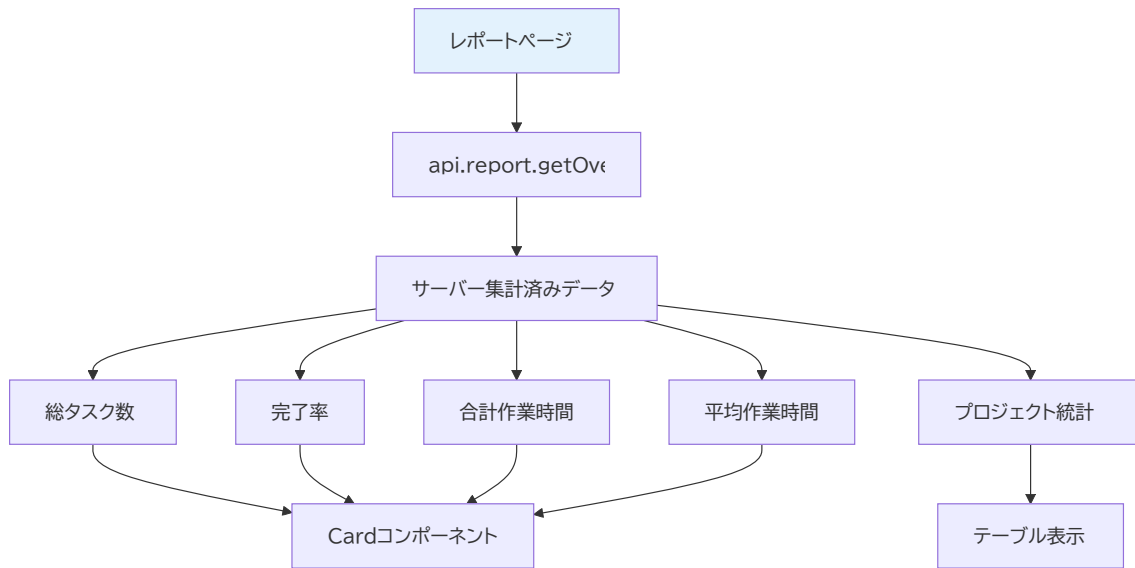


図 2: レポートページのデータフロー

この図では、画面から集計データを受け取る問い合わせを

`api.report.getOverview` の1つにまとめています。総タスク数・完了率・作業時間のどれも、同じ1回の問い合わせから枝分かれしています。カードごとに別々のAPIを呼ぶ形にすると読み込み中の判定が4つに増え、サーバー側の集計も4本に分かれます。数え方を直したいときも4か所を触ることになります。入口を1本にしておけば直す場所は `getOverview` だけです。通信の回数については4回になるとは限りません。このアプリは同時に呼ばれた問い合わせをまとめて1回で送る設定にしているためです。なお中の集計は `Promise.all` で同時に走るので実行の最中にタスクが増えると数字が1件ずれることはあります。

やること / やらないこと

やること	やらないこと
タスク総数の表示	画面側での再集計

Day 21: 統計カードを表示しよう

やること	やらないこと
完了率の計算	グラフ表示(Day 22)
作業時間の集計	週次レポート(Day 23)
Cardコンポーネント使用	専用コンポーネント作成

新しく学ぶ概念

概念	読み方	役割	例え
getOverview	ゲットオーバービュー	レポート用の集計済みAPI	店員さんが合計済みのレシートを渡す
groupBy	グループバイ	サーバー側で件数を集計	種類ごとに仕分けして数える
aggregate	アグリゲート	合計値をまとめて計算	レジで合計金額を出す
toFixed	トゥフィクスト	小数点の桁数を丸める	小数第1位まで表示

開発サーバーは前の Day から動かしたまま使います。止めてあるときは

`npm run dev` で起動してから `http://localhost:3000` を開きます。

実装ステップ一覧

ステップ	作業内容	読む時間の目安
Step 0	レポート集計 API (getOverview)を自分で書く	14分

ステップ	作業内容	読む時間の目安
Step 1	サーバー集計の考え方	3分
Step 2	import 文を書く	3分
Step 3	ページの骨組みとサイドバー導線を作る	8分
Step 4	データを取得する	3分
Step 5	受け取った概要データを 読む	5分
Step 6	ローディング判定を追加	3分
Step 7	統計カードを表示する	5分
Step 8	プロジェクト統計テーブル	5分
Step 9	動作確認	3分

読む時間の合計(仮): 約52分です。

表と各 Step に記した時間は、説明とコードを読む時間の仮の目安です。実測した値ではありません。コードの入力、動作確認、ダウンロードや起動の待ち時間、調べものには別に時間を取ってください。

Step 0: レポート集計 API(getOverview)を自分で書く(読む目安: 14分)

ゴール: `src/server/api/routers/report.ts` を新規作成し、`getOverview` を写経して `api.report.getOverview` を自分で生やします。
Day 09 の `project.getAll` や Day 13 の `task.getAll` と同じで、今日は「統計カードに渡す集計の入口」を1つ作ります。

統計カードは一覧データをクライアントで数え直しているわけではありません。完成版のコードは `count`・`aggregate`・`groupBy` を server 側にまとめ、画面には「計算済みの答え」だけを返します。件数が増えてもカードとテーブルの数字がぶれないようにするためです。

0-1. import を並べる

まず `src/server/api/routers/report.ts` を新規作成し、先頭に import を書きます。

```
// filepath: src/server/api/routers/report.ts
import { TASK_STATUS } from '@lib/constant/status';
import { prisma } from '@lib/prisma';
import { createTRPCRouter, protectedProcedure } from
'../trpc';
```

`TASK_STATUS` はこの後でステータス別に件数を数えるために使います。プロジェクト所属は、各集計クエリの `where` へ直接書きます。Day 23 で初めて使う `TRPCError`・`z`・`USER_ROLE` は未使用 import にしないため今日はまだ追加しません。

0-2. ログイン中のユーザー ID を受け取る

```
// filepath: src/server/api/routers/report.ts(続き)
export const reportRouter = createTRPCRouter({
  getOverview: protectedProcedure.query(async ({ ctx
})) => {
    const userId = ctx.session.userId;
```

`userId` は、この後の12本すべてで現在のプロジェクト所属を確かめるために使います。参加中のプロジェクトが0件なら、各クエリが0件や空配列を返すため、最後に同じ形の「空のレポート」が組み立てられます。

0-3. 集計の土台となる条件を作る

```
// filepath: src/server/api/routers/report.ts(続き)

// アーカイブ済みプロジェクトのタスクは集計対象外にし、
// プロジェクト数・統計との整合を取る。
const projectScope = {
  project: {
    isArchived: false,
    members: { some: { userId } },
  },
} as const;

// ダッシュボードの「アクティブな作業」を母数とするため、
// CANCELLED は集計から除外する。
const activeTasksFilter = {
  ...projectScope,
  NOT: { status: TASK_STATUS.CANCELLED },
} as const;
```

`projectScope` は「自分が参加中で、しかもアーカイブされていないプロジェクトの範囲」です。`project: { isArchived: false }` があるため参加しているプロジェクトを全部アーカイブしているとその範囲に入るタスクが1件も無くなります。Day 11 のアーカイブ操作で「Webサイトリニューアル」をアーカイブしたまま解除していない場合がこれに当たります。エラーは出ず、カードの数字だけが全部 0 になります。心当たりがあれば `/project` でアーカイブ表示を ON にして解除してから先

へ進んでください。`activeTasksFilter` はそこにさらに `CANCELLED` 除外を足した条件で、カードの母数に使います。こうして条件を変数にまとめておくと同じ条件を `count`・`groupBy` の全部で使い回せます。

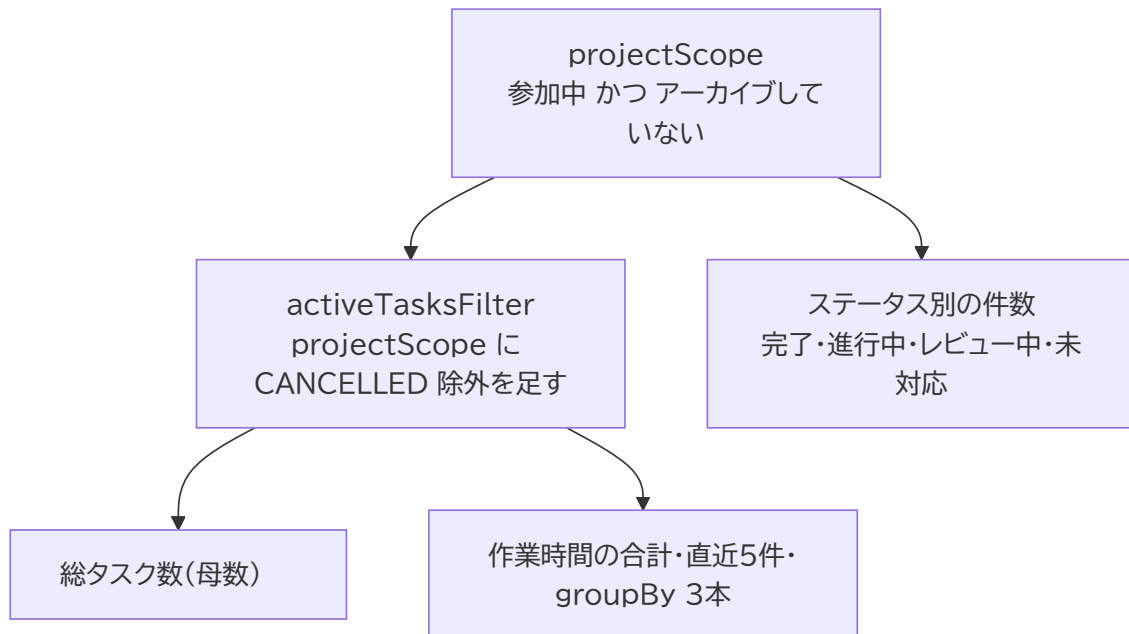


図3: 0-3. 集計の土台となる条件を作る

外側の条件を内側が取り込む入れ子です。完了件数が `projectScope` のままなのは取り消し済みのタスクはそもそも完了ではなく、除外しても件数が変わらないためです。母数の側だけが `CANCELLED` を外した条件を使います。

0-4. Promise.all で集計をまとめて取る

```
// filepath: src/server/api/routers/report.ts(続き)

const [
  projects,
  totalTasks,
  completedTasks,
  inProgressTasks,
  inReviewTasks,
  todoTasks,
  totalTimeAggregate,
  recentTasks,
  statusGroups,
  priorityGroups,
  projectTaskGroups,
  projectDoneGroups,
] = await Promise.all([
```

`Promise.all` は「独立した問い合わせをまとめて同時に待つ」書き方です。プロジェクト数、完了数、優先度別件数などは互いに依存しないので順番に12回待つよりまとめて走らせる方が素直です。

左辺に並べた12個の名前はこのあと書く問い合わせと同じ並びで受け取ります。1つ入れ替えると `completedTasks` に優先度別の配列が入ってしまいます。件数どうしの入れ替わりなら型も合ってしまうのでエラーは出ません。画面には見た目だけ正しい、中身の違う数字が並びます。あとで集計を足すときは左辺と問い合わせの両方の末尾へ足してください。

```
// filepath: src/server/api/routers/report.ts(続き)
prisma.project.findMany({
  where: {
    isArchived: false,
    members: { some: { userId } },
  },
  select: { id: true, name: true },
  orderBy: { createdAt: 'desc' },
}),
prisma.task.count({ where: activeTasksFilter }),
prisma.task.count({
  where: {
    ...projectScope,
    status: TASK_STATUS.DONE,
  },
}),
prisma.task.count({
  where: {
    ...projectScope,
    status: TASK_STATUS.IN_PROGRESS,
  },
}),
```

最初の数本は `findMany` と `count` です。ここでは「プロジェクト一覧」「総タスク数」「完了数」「進行中数」を取っています。`projectScope` と `activeTasksFilter` を使い分けている点が重要で、完了数などのステータス別件数は「アーカイブ済みではないプロジェクト内」で数え、総タスク数はさらに `CANCELLED` を除外した母数を使います。

Day 21: 統計カードを表示しよう

```
// filepath: src/server/api/routers/report.ts(続き)

prisma.task.count({
  where: {
    ...projectScope,
    status: TASK_STATUS.IN_REVIEW,
  },
}),
prisma.task.count({
  where: {
    ...projectScope,
    status: TASK_STATUS.TODO,
  },
}),
prisma.task.aggregate({
  where: activeTasksFilter,
  _sum: { timeSpentMinutes: true },
}),
```

aggregate は合計値をまとめて返す Prisma の関数です。ここでは **timeSpentMinutes** の合計だけが欲しいので **_sum** を使います。Day 16 で記録した作業時間を、ここでレポートの数字に変換します。

```
// filepath: src/server/api/routers/report.ts(続き)

prisma.task.findMany({
  where: activeTasksFilter,
  select: {
    id: true,
    title: true,
    status: true,
    priority: true,
  },
  orderBy: { updatedAt: 'desc' },
  take: 5,
}),
prisma.task.groupBy({
  by: ['status'],
  where: activeTasksFilter,
  _count: { _all: true },
}),
```

`recentTasks` は「直近で更新された5件」です。`groupBy` は「同じ値ごとにまとめて数える」関数で、ここでは `status` ごとの件数を作ります。Day 22 の円グラフはこの `statusData` に日本語ラベルと色を足して使います。

Day 21: 統計カードを表示しよう

```
// filepath: src/server/api/routers/report.ts(続き)

prisma.task.groupBy({
  by: ['priority'],
  where: activeTasksFilter,
  _count: { _all: true },
}),

prisma.task.groupBy({
  by: ['projectId'],
  where: activeTasksFilter,
  _count: { _all: true },
  _sum: { timeSpentMinutes: true },
}),

prisma.task.groupBy({
  by: ['projectId'],
  where: {
    ...projectScope,
    status: TASK_STATUS.DONE,
  },
  _count: { _all: true },
}),

]);
```

ここで「優先度別件数」「プロジェクトごとの総タスク数と合計時間」「プロジェクトごとの完了数」を取っています。後で `projectStats` を作るために、`groupBy` を2本に分けているのがポイントです。プロジェクト別の2本は同じIDでまとめますが、全タスクと完了タスクで集計条件が異なります。その結果を最後に組み合わせます。

0-5. 表示用データに組み立てて返す

```
// filepath: src/server/api/routers/report.ts(続き)
const totalTimeSpent =
totalTimeAggregate._sum.timeSpentMinutes ?? 0;
const averageTimePerTask = totalTasks > 0 ?
totalTimeSpent / totalTasks : 0;
const completionRate = totalTasks > 0 ? Math.round((
completedTasks / totalTasks) * 100) : 0;
```

この3行はカードに直接出す値を作る部分です。`?? 0` は「左が無いときだけ 0 を使う」書き方でした。タスクが0件のときに 0 で割ると壊れるので平均と完了率は三項演算子で先に守っています。

分母に置いているのは `totalTasks` です。この値は Step 0-3 の `activeTasksFilter` を通った件数なので中止したタスクとアーカイブ済みプロジェクトのタスクは最初から数に入っていません。どの行を除いたかで完了率は変わるため分母の正体は毎回確かめる価値があります。三項演算子で0件を先に弾いているのは `0 / 0` が `NaN` になるからです。守りを外すと完了率のカードには数字ではなく「NaN%」という文字が出ます。

Day 21: 統計カードを表示しよう

```
// filepath: src/server/api/routers/report.ts(続き)

const doneCountByProject = new Map(
  projectDoneGroups.map((group) => [group.projectId,
group._count._all]),
);

const taskStatsByProject = new Map(
  projectTaskGroups.map((group) => [
    group.projectId,
    {
      totalTasks: group._count._all,
      totalTimeSpent: group._sum.timeSpentMinutes ??
0,
    },
  ]),
);
```

Map は「projectId をキーにしてあとで素早く取り出す辞書」です。

projectDoneGroups と **projectTaskGroups** はどちらも **projectId** ごとの集計なのでいったん **Map** に変えてから **projects.map(...)** で合体させます。

```
// filepath: src/server/api/routers/report.ts(続き)
return {
  totalProjects: projects.length,
  totalTasks,
  completedTasks,
  inProgressTasks,
  inReviewTasks,
  todoTasks,
  completionRate,
  totalTimeSpent,
  averageTimePerTask,
  recentTasks,
  statusData: statusGroups.map((group) => ({
    key: group.status,
    value: group._count._all,
  })),
  priorityData: priorityGroups.map((group) => ({
    key: group.priority,
    value: group._count._all,
  })),
}
```

ここで `statusGroups` と `priorityGroups` を、画面が使いやすい `{ key, value }` 配列へ並べ替えています。`groupBy` の生データをそのまま返さず、UI がそのまま読みやすい形に直して返すのがポイントです。

Day 21: 統計カードを表示しよう

```
// filepath: src/server/api/routers/report.ts(続き)
projectStats: projects.map((project) => {
  const taskStats = taskStatsByProject.get(
project.id) ?? {
    totalTasks: 0,
    totalTimeSpent: 0,
  };
  const completedTaskCount = doneCountByProject.get(
project.id) ?? 0;

  return {
    id: project.id,
    name: project.name,
    totalTasks: taskStats.totalTasks,
    completedTasks: completedTaskCount,
    progress:
      taskStats.totalTasks > 0 ? (
completedTaskCount / taskStats.totalTasks) * 100 : 0,
    totalTimeHours: taskStats.totalTimeSpent / 60,
  };
}),
};
}),
});
```

最後の `projectStats` が Day 21 のテーブルに入る配列です。各プロジェクトについて `Map` から総数と完了数を取り出し、進捗率と時間を計算して返します。ここまで書けたら `getOverview` は完成です。

0-6. root.ts に時系列順で登録する

ルーターを書いただけではまだ `api.report.getOverview` とは呼べません。
`src/server/api/root.ts` に追加して初めてフロントから呼べる名前になります。Day 18 で書いた `root.ts` を開きます。import の並びと `appRouter` の中身を次の2ブロックの形に書き換えます。増えるのは `reportRouter` の import 1 行と `report: reportRouter,` の1行だけです。ほかの行は貼り足さずにそのまま残します。

```
// filepath: src/server/api/root.ts
import { authRouter } from './routers/auth';
import { commentRouter } from './routers/comment';
import { projectRouter } from './routers/project';
import { reportRouter } from './routers/report';
import { searchRouter } from './routers/search';
import { taskRouter } from './routers/task';
import { createCallerFactory, createTRPCRouter } from
'./trpc';
```

先頭の import はこれまで作ってきた router の名前をこのファイルへ持ち込むだけです。名前が来ただけではまだ `api.report.getOverview` とはつながりません。ここで手を止めて画面から呼ぶと `api.report` の部分で「そんなプロパティは無い」という型エラーが出ます。import の並びは Biome の整列に合わせています。

Day 21: 統計カードを表示しよう

```
// filepath: src/server/api/root.ts(続き)
export const appRouter = createTRPCRouter({
  auth: authRouter,
  project: projectRouter,
  task: taskRouter,
  search: searchRouter,
  comment: commentRouter,
  report: reportRouter,
});
```

`appRouter` 中の順番は教材で作ってきた時系列に揃えます。Day 21 の時点では `report` が `comment` の次にある最後の router です。`user` は Day 24 でファイルを作ってから追加します。

確認ポイント:

- `src/server/api/routers/report.ts` を新規作成し、`getOverview` を最後の `});` まで書きました
- `root.ts` に `reportRouter` を追加し、`report: reportRouter` を時系列順で登録しました
- `npx tsc --noEmit` で型エラーが出ていません

Step 1: サーバー集計の考え方(読む目安: 3分)

ゴール: なぜ完成版のコードでは専用の集計APIを使うのかを理解します。

2つの集計方法の比較

方法	仕組み	メリット	デメリット
サーバー集計	APIが計算済み値を返す	件数が増えても正確・高速	API設計が必要
ローカル集計	生データから計算	試作は早い	一覧APIの件数上限に引きずられやすい

初期案では `api.task.getAll` と `api.project.getAll` をクライアントで集計することもできますが完成版のコードは `api.report.getOverview` に統合しています。これは「100件を超えても統計が欠けない」状態を守るためです。

`getOverview` が返すもの

`api.report.getOverview` は完成版のコードで必要な集計をサーバー側でまとめて返します。クライアントは「計算する側」ではなく「受け取って表示する側」に集中します。

プロパティ	内容
<code>totalTasks</code>	集計対象タスク数(キャンセル済みタスクとアーカイブ済みプロジェクトは除く)
<code>completionRate</code>	完了率(整数パーセント)
<code>totalTimeSpent</code>	合計作業時間(分)
<code>averageTimePerTask</code>	1タスクあたり平均作業時間(分)

プロパティ	内容
-------	----

`projectStats`

プロジェクト別の集計済み配列

```
// filepath: src/app/report/page.tsx
// Step 3 以降でこの API を実際に呼び出します
api.report.getOverview.useQuery();
```

実際に書くのは Step 4 です。ここでは形だけ見ておきます。かっこの中が空なのは誰の集計を出すかを画面が指定しないからです。対象のユーザーは server 側が `ctx.session` から取り出すのでブラウザから他人のIDを渡しても覗けません。Day 09 の `getAll` では `userId` を受け取ったうえで管理者だけに許しましたが今日は「そもそも受け取らない」形で同じ守りを掛けています。

確認ポイント:

- 完成版のコードがサーバー集計を選んだ理由を理解しました
- 一覧APIと統計APIは責務を分けるべきだと理解しました

Step 2: import 文を書く(読む目安: 3分)

ゴール: 必要なモジュールを読み込みます。

まず `src/app/report/page.tsx` を新規作成し、先頭に以下の import を書きます。

実装:

```
// filepath: src/app/report/page.tsx
'use client';

import { AppLayout }
  from '@component/layout/app-layout';
```

'use client' は、このファイルをクライアントコンポーネントの入口にする宣言です。Step 4 で使う tRPC の `useQuery` はこの境界の内側で呼びます。宣言を忘れると、サーバーコンポーネントから呼べない機能を使うためエラーになります。初回の HTML はクライアントコンポーネントもサーバーで生成されます。`AppLayout` は Day 08 で仕上げた共通の枠で、これで囲んだページにはサイドバーとヘッダーが自動で付きます。

確認ポイント:

- ファイルを新規作成しました
- 'use client' を先頭に書きました

Day 21: 統計カードを表示しよう

```
// filepath: src/app/report/page.tsx
// shadcn/ui のカード部品
import {
  Card, CardContent,
  CardHeader, CardTitle,
} from '@component/ui/card';
// ローディング表示
import { PageLoadingSpinner }
  from '@component/ui/loading-spinner';
```

`Card` は shadcn/ui のカード部品です。`CardHeader` が見出しの帯、`CardContent` が中身の入れ物、`CardTitle` が見出しの文字を受け持ちます。統計カードは4枚とも同じ見た目にそろえたいので枠線や角丸を自分で書かず、すでにある部品を使います。`PageLoadingSpinner` は Day 09 の一覧画面でも出した読み込み中の表示です。集計は件数が増えるほど返るまでが長くなるので待っている間に見せる画面を先に手元へ用意しておきます。

確認ポイント:

- `Card` 関連をインポートしました
- `PageLoadingSpinner` をインポートしました

```
// filepath: src/app/report/page.tsx
// テーブル部品(プロジェクト統計用)
import {
  Table, TableBody, TableCell,
  TableHead, TableHeader, TableRow,
} from '@component/ui/table';
```

テーブルは6つの部品に分かれています。`Table` が外枠、`TableHeader` と `TableBody` が見出し行と本体行のまとめ、`TableRow` が1行、`TableHead` が見出しのセル、`TableCell` が中身のセルです。素の `<table>` タグでも表は組めますがこの部品を通すと罫線の色や文字の大きさがアプリの他の画面とそろいます。6つ全部を Step 8 のプロジェクト統計テーブルで使うのでここでまとめて読み込んでおきます。

確認ポイント:

- テーブル関連の部品をインポートしました

```
// filepath: src/app/report/page.tsx
// APIクライアント
import { api } from '@trpc/react';
```

`api` は tRPC のクライアントで、これを通すとサーバー側の手続きをただの関数のように呼べます。`api.report.getOverview` という呼び名が使えるのは Step 0-6 で `root.ts` に `report: reportRouter` を登録したからです。登録を飛ばしているとこの import 自体は通るのに `api.report` のところで型エラーが出ます。エディタで `api.` と打ったとき候補に `report` が出てこないなら Step 0-6 に戻ってください。

確認ポイント:

- `api` をインポートしました
- 保存してエラーが出ないこと

Step 3: ページの骨組みを作る(読む目安: 5分)

ゴール: ReportPage コンポーネントの骨組みを作ります。サイドバーの「レポート」をクリックして表示を確認します。

この時点では中身はまだ空です。見出しと説明文だけが表示されます。

実装:

```
// filepath: src/app/report/page.tsx
// コンポーネント本体(骨組み)
export default function ReportPage() {
  // Step 4~6 でここにフックを追加

  return (
    <AppLayout>
      <div className="space-y-6">
        <div>
          <h1 className="text-3xl
            font-bold tracking-tight">
            レポート・統計
          </h1>

```

数字を出す前に、空の器だけを置いて表示を確かめます。この段階で `/report` が開くと分かっているらあとでカードが出ないときに「ページ自体が無い」のか「データが来っていない」のかを切り分けられます。 `export default` を付けた関数が

Next.js ではそのURLの画面そのものになります。`AppLayout` で囲んでいるのでサイドバーとヘッダーは書かなくても付いてきます。`space-y-6` はこの中に縦へ並べる要素の間隔をそろえる指定です。

確認ポイント:

- 関数コンポーネントを定義しました
- `AppLayout` で囲みました

```
{/* filepath: src/app/report/page.tsx */}
{/* 骨組み続き: 説明文と閉じタグ */}
    <p className=
      "text-muted-foreground">
      プロジェクトの進捗とタスクの状況を確認できます。
    </p>
  </div>
  {/* Step 7~8 でカード等を追加 */}
</div>
</AppLayout>
);
}
```

開いていたタグを内側から順に閉じて骨組みが完成します。

`{/* Step 7~8 でカード等を追加 */}` は目印のコメントで、あとで JSX を差し込む場所を見失わないために置いています。ここで保存すると見出しと説明文だけのページが出ます。数字は1つも並びませんがそれで正常です。データを取りに行く処理は次の Step 4 で、このコメントより上へ足していきます。

Day 21: 統計カードを表示しよう

確認ポイント:

- `/report` にアクセスして表示されます
- 見出しと説明文が表示されます

骨組み確認: 下の画像は Step 8 まで書き終えた完成後の画面です。赤枠の中がこの Step で出るところです。いまの自分の画面にはその下のカードとテーブルがまだ出ていません。カードは Step 7、テーブルは Step 8 で足します。



図 4: 完成後のレポートページ。赤枠の中に、見出し「レポート・統計」と説明文の 2 行が出ている

レポートページが表示できたので、Day 08 で作ったサイドバーから移動できるようにします。`src/component/layout/app-layout.tsx` の `lucide-react` の import に `BarChart` を追加してください。

```
// filepath: src/component/layout/app-layout.tsx
import {
  BarChart,
  ClipboardList,
  FolderOpen,
  LayoutDashboard,
  ListTodo,
  LogOut,
  Search,
} from 'lucide-react';
```

足したのは `BarChart` の1語だけです。これで `lucide-react` から棒グラフのアイコンを1つ持ってこられます。他の6つはDay 20までに並べたものなので消さずにそのまま残してください。アルファベット順にそろえているのはBiomeのimport 整列に合わせるためです。順番が違くと保存したときに自動で並べ替えられ、変更していない行まで差分に入ります。

同じファイルの `menuItems` に、レポート用の項目を追加します。Day 20 までの項目を残し、配列の閉じカッコ直前へ入れます。

```
// filepath: src/component/layout/app-layout.tsx
{
  text: 'レポート',
  icon: <BarChart className="h-5 w-5" />,
  path: '/report',
},
```

Day 21: 統計カードを表示しよう

`menuItems` はサイドバーに並べるリンクの配列です。1つの要素が1行のメニューに対応し、`text` が表示名、`icon` が左に置くアイコン、`path` が飛び先のURLになります。ここに書いた `/report` は先ほど作ったファイルの場所と同じ綴りでなければいけません。ずれているとメニューは出るのに押した先が404になります。配列の末尾へ入れるのは教材で作ってきた順にメニューが並ぶようにするためです。

確認ポイント:

- サイドバーに「レポート」が表示されました
- 既存のメニュー項目が消えていません

Step 4: データを取得する(読む目安: 3分)

ゴール: tRPC の `getOverview` で集計済みデータをまとめて取得します。

配置場所: Step 3 のコメント `// Step 4~6 でここにフックを追加` の位置に追加します。`return` 文の前です。

実装:

```
// filepath: src/app/report/page.tsx
// ReportPage 内、return 文の前に追加
const { data: overview, isLoading } =
  api.report.getOverview.useQuery();
```

この1行で、カード4枚とテーブル1つに要る数字がまとめて手に入ります。`data` に `overview` という別名を付けているのは `data` のままでは何のデータか読み取れないからです。`isLoading` は最初の取得が終わるまで `true` で、Step 6 のス

ピナー表示に使用します。もし4枚のカードを別々の `useQuery` で作ると読み込み中の判定も4つに増え、カードが1枚ずつバラバラに出る画面になります。取得を1本にしておけば待ち状態も1つで済みます。

`getOverview` の中では `count` `aggregate` `groupBy` が使われ、100件を超えるデータでもサーバー側で正しい統計が作られます。

確認ポイント:

- `getOverview` 1つで統計データをまとめて取得しています
- 保存してエラーが出ないこと

Step 5: 受け取った概要データを読む(読む目安: 5分)

ゴール: `overview` のどのプロパティをどのカードに使うか整理します。

配置場所: Step 4 の `useQuery` の直後に続けて追加します。

Day 21: 統計カードを表示しよう

実装:

```
// filepath: src/app/report/page.tsx
// JSX で使う主な値
const totalTasks = overview?.totalTasks ?? 0;
const completionRate = overview?.completionRate ?? 0;
const totalTimeHours =
  ((overview?.totalTimeSpent ?? 0) / 60).toFixed(1);
const averageTimeHours =
  ((overview?.averageTimePerTask ?? 0) / 60).toFixed(1);
```

4 つとも `?? 0` で受け止めているのは `overview` が `undefined` になる場面があるからです。`undefined / 60` の答えは `NaN` で、`toFixed(1)` を通しても `NaN` のままです。画面には数字ではなく「NaNh」という文字が出ます。

この4行はStep 6 の読み込み判定より前に置きます。だから読み込み中でも計算そのものは走り、`?? 0` が効いて 0 になります。それでも画面へは出ません。Step 6 が先に読み込み中の表示を返すためです。`?? 0` の結果が実際に画面へ出るのは取得そのものが失敗したときです。ただしその場合画面には失敗した事実ではなく「0」が並びます。本当に0件なのか取得に失敗したのかを読者が見分けられないので実務では `error` も受け取って失敗したときだけ別の案内を出す形にします。この通信エラー表示は Day 26 で追加します。`totalTasks` と `completionRate` は server が出した答えをそのまま受け取るだけで、画面側では足し算や割り算を一切していません。

完成版のコードでは `overview.totalTimeSpent` と `overview.averageTimePerTask` は分単位で返るので表示時だけ `/ 60` して `toFixed(1)` で時間表記にします。

確認ポイント:

- 4 枚のカードが `overview` を元に表示されます
- 集計計算をクライアント側で書いていません

各統計値の計算ロジック

統計値	計算方法	使う関数
総タスク数	<code>overview.totalTasks</code>	server 集計結果
完了率	<code>overview.completionRate</code>	server 集計結果
合計時間	<code>overview.totalTimeSpent / 60</code>	表示時だけ時間 換算
平均時間	<code>overview.averageTimePerTask / 60</code>	表示時だけ時間 換算

Step 6: ローディング判定を追加(読む目安: 3分)

ゴール: データ取得中にスピナーを表示する `early return` を追加します。

配置場所: Step 4~5 のデータ取得・値の準備の下、`return` 文の前に追加します。

Day 21: 統計カードを表示しよう

実装:

```
// filepath: src/app/report/page.tsx
// 値の準備の下、return 文の前に追加
if (isLoading) {
  return <PageLoadingSpinner />;
}
```

early return とは条件を満たしたら本来の表示(カード等)を返さず、先にスピナーを返して処理を終える書き方です。

確認ポイント:

- ローディング中にスピナーが表示されます
- `getOverview` の結果待ちだけを判定しています

ローディング確認: 読み込んでいる間は画面の中央にスピナーだけが出ます。

ここでスピナーだけを返すと読み込み中はサイドバーも消えます。この教材ではページ全体の描画にデータが要る画面はこの形にしています。サイドバーを残したい場合は Day 17 のように `<AppLayout>` の内側でスピナーを返します。



図5: 読み込み中のレポートページ。サイドバーも消え、画面の中央に丸いスピナーだけが出ている

Step 7: 統計カードを表示する(読む目安: 5分)

ゴール: 4枚のカードで統計を表示します。

以下の JSX は Step 3 の `return` 内、コメント

`{/* Step 7~8 でカード等を追加 */}` の位置に追加します。

Day 21: 統計カードを表示しよう

実装:

```
{/* filepath: src/app/report/page.tsx */}
{/* 統計カード: タスク数と完了率 */}
<div className="grid grid-cols-1
  sm:grid-cols-2 lg:grid-cols-4
  gap-4">
  <Card>
    <CardContent className="pt-6">
      <p className="text-sm
        text-muted-foreground mb-1">
        タスク数</p>
      <p className="text-3xl font-bold">
        {totalTasks}</p>
    </CardContent>
  </Card>
```

1枚目に出る `totalTasks` はデータベースにあるタスクの全件数ではありません。Step 0-3 の `activeTasksFilter` を通った件数、つまりアーカイブ済みプロジェクトのタスクと `CANCELLED` のタスクを外した数です。中止を母数に残すとどうなるか、10件のうち5件を終えて3件を中止したプロジェクトで考えます。母数7件なら完了率は71%、中止も数えて母数10件にすると50%になります。手を動かした本人から見れば後者は「やめたはずの仕事に足を引っ張られた数字」に映ります。

確認ポイント:

- グリッドの開始タグを書きました

- 1枚目のカードを書いた(表示の確認は4ブロック目で `</div>` を閉じてから。開始タグだけの状態で保存すると構文エラー)

```
{/* filepath: src/app/report/page.tsx */}
{/* 統計カード: 完了率カード */}
<Card>
  <CardContent className="pt-6">
    <p className="text-sm
      text-muted-foreground mb-1">
      完了率</p>
    <p className="text-3xl font-bold">
      {completionRate}%</p>
  </CardContent>
</Card>
```

`completionRate` は server 側で `Math.round` を通した整数なので画面では `%` を付けるだけで済みます。Day 22 の円グラフが使うのはステータス別・優先度別の件数で、完了率は表示しません。完了率カードは整数、プロジェクト統計テーブルの進捗は小数第1位と、表示先に合わせて精度を分けています。分子は `DONE` のタスク数、分母は中止を除いた件数です。中止したタスクが `DONE` になることはないのので分子には最初から入りません。分母からだけ消えます。

確認ポイント:

- 2枚目に `completionRate` と `%` を書きました
- まだ `</div>` を閉じていないので表示の確認は4ブロック目のあとに回します

Day 21: 統計カードを表示しよう

```
{/* filepath: src/app/report/page.tsx */}
{/* 統計カード: 合計と平均の作業時間 */}
<Card>
  <CardContent className="pt-6">
    <p className="text-sm
      text-muted-foreground mb-1">
      合計作業時間</p>
    <p className="text-3xl font-bold">
      {totalTimeHours}h</p>
  </CardContent>
</Card>
```

`totalTimeHours` は Step 5 で作った文字列です。server は分で返し、画面へ出す直前だけ60で割ります。分のまま持ち回るほうが時間と分の取り違えを防げるからです。割り忘れると480分の作業が「480h」と表示されます。20日ぶん働いた計算になるので見た瞬間におかしいと気付けます。合計する対象はここでも中止を除いたタスクです。やめた作業に費やした時間は進み具合の目安から外します。

確認ポイント:

- 分を時間に変換(÷60)しています
- `toFixed(1)` で小数1桁に丸めています

```

{/* filepath: src/app/report/page.tsx */}
{/* 統計カード: 平均作業時間 + grid閉じ */}
  <Card>
    <CardContent className="pt-6">
      <p className="text-sm
        text-muted-foreground mb-1">
        平均作業時間/タスク</p>
      <p className="text-3xl font-bold">
        {averageTimeHours}h</p>
    </CardContent>
  </Card>
</div>

```

4枚目の平均作業時間はserver側で合計時間を `totalTasks` で割った値です。分母がここでも中止を除いた件数なのでやめたタスクが平均を薄めることはありません。最後の `</div>` はこのステップの先頭で開いたグリッドを閉じるタグです。閉じ忘れると開始タグと終了タグの対応が取れなくなり、レイアウトが崩れる前にコンパイル自体が失敗します。ブラウザには統計カードではなく構文エラーの画面が出てターミナルには問題のファイル名と行番号が並びます。カードが1枚も出ないときはまずこの1行を確かめてください。

確認ポイント:

- 4枚のカードが表示されます
- 正しい数値が表示されます

Day 21: 統計カードを表示しよう

カード確認: 下の画像は Step 8 まで書き終えた完成後の画面です。赤枠の中がこの Step で足した4枚のカードです。いまの自分の画面にはその下のプロジェクト統計テーブルがまだ出ていません。テーブルは Step 8 で足します。



図 6: 完成後のレポートページ。赤枠の中に、タスク数・完了率・合計作業時間・平均作業時間の4枚のカードが横に並んでいる

Step 8: プロジェクト統計テーブル(読む目安: 5分)

ゴール: プロジェクトごとの統計をテーブルで表示します。

完成版のコードではプロジェクト別集計も `overview.projectStats` に入って返ってきます。

まずこれから書く行の中身を1つの塊で見ておきます。このブロックはまだ書きません。実際に書く場所はこのあと作る `<TableBody>` の中です。

```

{ /* filepath: 読み比べ用サンプル(実ファイルには対応しませ
ん) */}
{ /* projectStats は server 側で集計済み */}
{overview?.projectStats.map((stat) => (
  <TableRow key={stat.id}>
    <TableCell className="font-medium">
      {stat.name}
    </TableCell>
    <TableCell className="text-right">
      {stat.totalTasks}
    </TableCell>
    <TableCell className="text-right">
      {stat.completedTasks}
    </TableCell>
    <TableCell className="text-right">
      {stat.progress.toFixed(1)}%
    </TableCell>
    <TableCell className="text-right">
      {stat.totalTimeHours.toFixed(1)}h
    </TableCell>
  </TableRow>
)}}

```

この `map` が回すのは Step 0-5 で `Map` から組み立てた `projectStats` です。画面側では `filter` や `reduce` を呼びません。プロジェクトごとの完了数をここで数え直すとカードの数字と表の数字が別々の計算から出ることになります。除外

Day 21: 統計カードを表示しよう

の条件を片方だけ直したとき合計と内訳の合わない表ができあがります。

`stat.progress` と `stat.totalTimeHours` に `toFixed(1)` を付けているのは `71.42857142857143` のような値をそのまま出さないためです。

確認ポイント:

- `overview.projectStats` をそのまま描画しています
- クライアント側で `filter` / `reduce` を再実行していません

次に Step 7 のカードグリッドの `</div>` の直後にテーブルの JSX を追加します。

```
{/* filepath: src/app/report/page.tsx */}
{/* テーブル: ヘッダー部分 */}
<Card>
  <CardHeader>
    <CardTitle>
      プロジェクト統計</CardTitle>
    </CardHeader>
    <CardContent>
      <Table>
        <TableHeader>
          <TableRow>
            <TableHead className="w-[200px]">
              プロジェクト</TableHead>
            <TableHead className="text-right">
              タスク数</TableHead>
            </TableRow>
          </TableHeader>
        </Table>
      </CardContent>
    </CardHeader>
  </Card>
```

テーブルもカードと同じ `Card` の中に置きます。見出しと表を1つの枠にまとめると上の4枚と同じ余白・同じ角丸で並び、レポートページ全体が1つの面に見えます。

`TableHeader` の中の `TableRow` は見出しの行で、あとで書く本体の行とは別のまとまりです。先頭の列だけ `w-[200px]` で幅を決めているのはプロジェクト名の長さがまちまちでも、右側の数字の列を行ごとにずらさないためです。

確認ポイント:

- `Card` の中に `Table` を配置しています
- ヘッダー行を書きました

```
{/* filepath: src/app/report/page.tsx */}
{/* テーブル: ヘッダー残りと閉じタグ */}
    <TableHead className="text-right">
      完了</TableHead>
    <TableHead className="text-right">
      進捗</TableHead>
    <TableHead className="text-right">
      作業時間</TableHead>
  </TableRow>
</TableHeader>
```

残り3つの見出しを足して5列がそろいます。数字の列に `text-right` を付けているのは桁数の違う数字を右端でそろえるためです。`7` と `123` を左寄せで並べると一の位の位置がずれ、どちらが大きいかわかりません。ここで `</TableRow>` と `</TableHeader>` を閉じて見出しのまとまりを終わらせます。次のブロックからプロジェクト1件が1行になる本体へ入ります。

Day 21: 統計カードを表示しよう

確認ポイント:

- 5列のヘッダーが揃いました
- 次のブロックで行データを追加します

```
{/* filepath: src/app/report/page.tsx */}
{/* テーブル: 行データと閉じタグ */}
<TableBody>
  {overview?.projectStats.map((stat) => (
    <TableRow key={stat.id}>
      <TableCell
        className="font-medium">
        {stat.name}</TableCell>
      <TableCell
        className="text-right">
        {stat.totalTasks}</TableCell>
      <TableCell
        className="text-right">
        {stat.completedTasks}</TableCell>
```

`TableBody` の中で `projectStats` を1件ずつ行に変えます。`key` に `stat.id` を渡しているのはReact が再描画のときにどの行がどのプロジェクトかを見分けるためです。ここを配列の番号にするとReact は「0番目の行はこれからも0番目の行だ」と判断して同じ要素を使い回し、中の文字だけを差し替えます。`projectStats` は作成日の新しい順に並んでいるのでプロジェクトを1つ作ると先頭に割り込み、後ろの番号が1つずつずれます。するとさっきまで別のプロジェクトを映していた行に、隣のプロジェクトの数字が流し込まれます。行に入力欄や☐チェック

ボックスを足したときはそこへ入れた値まで別のプロジェクトの行へ移ります。ID を渡しておけば番号がずれても React は同じプロジェクトの行を追いかけてくれます。先頭の `overview?.projectStats` にある `?.` は `overview` がまだ届いていない一瞬を素通りさせる書き方です。

確認ポイント:

- `map` でプロジェクトごとに行を生成
- `key` にプロジェクトIDを指定

```
{/* filepath: src/app/report/page.tsx */}
{/* テーブル: 残り列と全閉じタグ */}
    <TableCell
      className="text-right">
      {stat.progress.toFixed(1)}%</TableCell>
    <TableCell
      className="text-right">
      {stat.totalTimeHours.toFixed(1)}h
    </TableCell>
  </TableRow>
)}
</TableBody>
</Table>
</CardContent>
</Card>
```

Day 21: 統計カードを表示しよう

残り2列を足して開いていたタグを内側から順に閉じます。`TableRow`、`TableBody`、`Table`、`CardContent`、`Card` と、開いた順の逆にたどるのが JSX の決まりです。1つでも順番を入れ違えると保存した瞬間に閉じタグが合わないという構文エラーが出て画面がエラー表示に切り替わります。これで4枚のカードの下に内訳の表が並び、合計がどのプロジェクトから来ているかを追えるようになります。

確認ポイント:

- プロジェクト統計テーブルが表示されます
- 名前・タスク数・完了数・進捗・時間が並びます

Step 9: 動作確認(読む目安: 3分)

ゴール: 統計カードの表示を確認します。

開発サーバーが動いていればそのまま使います。止めてあるときだけ次のコマンドで起動します。

```
# filepath: ターミナル(確認用)
# 停止している場合だけ起動する
npm run dev
# http://localhost:3000/report にアクセス
```

開発サーバーが動いていればそのまま使います。起動したらまずカードの数字がシードデータと合っているかを確認めます。総タスク数が初期データの5件より少なくても正常です。ログイン中のユーザーが参加しているプロジェクトのタスクだけを数えるためです。管理者アカウントは2つあるプロジェクトのうち1つにしか参加していない

ので3件と表示されます。この3件は初期データを触っていない場合の数です。Day 14 で自分がタスクを作っていればその分だけ増えます。数が違って実装の誤りではありません。ただし参加していないプロジェクトのタスクまで数に入っているようならStep 0-3 の `activeTasksFilter` をどこかの `count` で使い忘れています。4枚とも0でテーブルも空なら、まず参加中のアクティブなプロジェクトにタスクがあるかを確認します。プロジェクトのアーカイブは原因の1つです。対象があるのに0のままなら、ブラウザのコンソールと開発サーバーのターミナルで通信やサーバー処理のエラーを確認してください。

ブラウザの DevTools を開き(`F12` キー)、画面幅を変更してカードの並びを確認します。

1. `/report` にアクセス
2. 4枚のカードが表示されます
3. 総タスク数がキャンセル済みを除いたタスク件数と一致
4. 完了率が正しく計算されています
5. 作業時間が時間(`h`)で表示されます
6. ブラウザ幅を変えてレスポンス確認

グリッドのブレイクポイント

画面サイズ	クラス	列数
モバイル	<code>grid-cols-1</code>	1列
タブレット	<code>sm:grid-cols-2</code>	2列
PC	<code>lg:grid-cols-4</code>	4列

Day 09 のプロジェクト一覧や Day 13 のタスク一覧で使ったレスポンシブグリッドと同じパターンです。

確認ポイント:

- 数値がシードデータと一致します
- カードが正しくグリッド表示されます
- ブラウザ幅を変えると列数が変わります

レスポンシブ確認: ブラウザの幅を狭めていくとカードの並びが4列から2列、1列へと変わります。下の画像はブラウザの幅を 900px まで縮めたときの姿で、

`lg:grid-cols-4` が外れて `sm:grid-cols-2` が効いています。



図 7: 幅を狭めたレポートページ。統計カードが2列2段に折り返して並んでいる

Pro パターンで書こう(統計レイアウトの Server/Client 分離)

Before(改善前のコード)

```
// filepath: 読み比べ用サンプル(参考・実ファイルには対応しません)

"use client";

export default function ReportPage() {

  const { data } = api.report.getOverview.useQuery();

  return (

    <AppLayout>

      <h1>レポート</h1>

      <div className="grid grid-cols-4 gap-4">

        <StatCard title="総タスク" value={

data?.totalTasks ?? 0} />

        <StatCard title="完了" value={

data?.completedTasks ?? 0} />

      </div>

    </AppLayout>

  );

}
```

この書き方でも、Next.js は初回表示用の HTML をサーバーで作ります。`"use client"` は HTML を作る場所だけを決める宣言ではありません。このファイルと取り込んだ部品の JavaScript をブラウザへ送る境界です。静的な見出しも同じファイルにあるためその表示コードまで送られます。

このコードの問題点:

- 静的な見出しのコードもブラウザへ送られます
- 表示だけの部分と、データ取得や操作を担う部分が同じファイルにあります

After(プロが書くコード)

```
// filepath: 読み比べ用サンプル(参考・実ファイルには対応しません)
// page.tsx (Server Component)
export default function ReportPage() {
  return (
    <AppLayout>
      <h1>レポート</h1>
      <ReportContent />
    </AppLayout>
  );
}

// report-content.tsx ("use client")
"use client";
export function ReportContent() {
  const { data } = api.report.getOverview.useQuery();
  return (
    <div className="grid grid-cols-4 gap-4">
      <StatCard title="総タスク" value={data?.totalTasks}
        ?? 0 />
      <StatCard title="完了" value={data?.completedTasks}
        ?? 0 />
    </div>
  );
}
```

分けたあとの `page.tsx` には `"use client"` がありません。見出しを組み立てるコードはサーバーに残り、`useQuery` を使う `ReportContent` はブラウザでも実行されます。`AppLayout` は独自に `"use client"` を持つ部品なので分割後もその JavaScript は必要です。初回の HTML は分割の有無にかかわらずサーバーで作られます。

このコードの強み:

- 静的な見出しを組み立てるコードをサーバー側に残せます
- データ取得を担う `ReportContent` を別ファイルで読めます
- ブラウザへ送るコードの範囲を必要な部分に絞れます

覚えておきたいエッセンス

クリックや入力、ブラウザ用のフックが必要な部分に `"use client"` を置きます。Server Component は静的な表示に加え、サーバー側でのデータ取得も担当できます。

完成コード全体

今日は4つのファイルを変更しました。`report.ts`・`root.ts`・レポートの `page.tsx` は、各見出しの下のコードブロックを順につなげると全文になります。途中で貼る位置が分からなくなった場合は、この3ファイルを完成コードと見比べてください。

`app-layout.tsx` は変更した2か所だけの抜粋です。ファイル全体を置き換えず、該当する import とメニュー項目を更新してください。Day 20 までに追加した内容も残します。

ファイル	役割	対応する Step
<code>src/server/api/routers/report.ts</code>	集計済みの数値を返す <code>getOverview</code>	Step 0
<code>src/server/api/root.ts</code>	手続きの一覧表への <code>report</code> の登録	Step 0
<code>src/app/report/page.tsx</code>	統計カードとプロジェクト統計の画面	Step 2～ Step 8
<code>src/component/layout/app-layout.tsx</code>	サイドバーからレポートページへの入口	Step 2

`src/server/api/routers/report.ts`

インポート:

```
// filepath: src/server/api/routers/report.ts
// 完成版: インポート
import { TASK_STATUS } from '@lib/constant/status';
import { prisma } from '@lib/prisma';
import { createTRPCRouter, protectedProcedure } from
'../trpc';
```

`TASK_STATUS` を取り込んでいるので集計の条件に `'DONE'` という文字列を直接書かずに済みます。文字列で書くと打ち間違えても動いてしまい、完了件数が常に0になる不具合として現れます。定数なら打ち間違いは編集集中に分かります。

各クエリに `members: { some: { userId } }` を含め、集計の対象を取得時点で自分が参加しているプロジェクトへ絞ります。

getOverview の入口:

```
// filepath: src/server/api/routers/report.ts(同じファイルの続き)
// 完成版: getOverview の入口と空のときの戻り値
export const reportRouter = createTRPCRouter({
  getOverview: protectedProcedure.query(async ({ ctx
})) => {
  const userId = ctx.session.userId;
```

入口ではログイン中の `userId` を受け取ります。参加しているプロジェクトが0件なら、後続の各集計が0または空配列を返します。最後の戻り値は同じ13項目を保つため、画面側はそのまま「空のレポート」を描画できます。

集計の土台となる条件:

Day 21: 統計カードを表示しよう

```
// filepath: src/server/api/routers/report.ts(同じファイルの続き)
// 完成版: 集計の土台となる条件

// アーカイブ済みプロジェクトのタスクは集計対象外にし、
// プロジェクト数・統計との整合を取る。
const projectScope = {
  project: {
    isArchived: false,
    members: { some: { userId } },
  },
} as const;

// ダッシュボードの「アクティブな作業」を母数とするため、
// CANCELLED は集計から除外する。
const activeTasksFilter = {
  ...projectScope,
  NOT: { status: TASK_STATUS.CANCELLED },
} as const;
```

2つの条件を変数にしてあるのがこの手続きの数字を合わせている部分です。この後に12本の問い合わせが並びますが全部がこの2つのどちらかを使います。条件を毎回書き写すと1か所だけ書き漏らした問い合わせが別の母数で数え、画面上で合計が合わなくなります。

`activeTasksFilter` が `projectScope` を展開してから `NOT` を足しているので「アーカイブ済みを除く」と「取り消し済みを除く」が積み重なります。`as const` を付けているのはこの条件を書き換えられない形で固定するためです。

Promise.all の受け取り:

```
// filepath: src/server/api/routers/report.ts(同じファイルの続き)
// 完成版: Promise.all の受け取り

const [
  projects,
  totalTasks,
  completedTasks,
  inProgressTasks,
  inReviewTasks,
  todoTasks,
  totalTimeAggregate,
  recentTasks,
  statusGroups,
  priorityGroups,
  projectTaskGroups,
  projectDoneGroups,
] = await Promise.all([
```

左側の12個の名前と、この後に並ぶ12本の問い合わせは書いた順番どおりに対応します。名前を1つ入れ替えると件数の変数に作業時間が入り、型が合う場合はエラーも出ません。数字だけが静かに入れ替わるので追加や並べ替えのときは上下を見比べてください。

`Promise.all` にしているので12本は同時に走ります。順番に `await` すると1本ずつの待ち時間が積み上がります。

プロジェクトと件数の集計・前半:

Day 21: 統計カードを表示しよう

```
// filepath: src/server/api/routers/report.ts(同じファイルの続き)
// 完成版: プロジェクトと件数の集計・前半

prisma.project.findMany({
  where: {
    isArchived: false,
    members: { some: { userId } },
  },
  select: { id: true, name: true },
  orderBy: { createdAt: 'desc' },
}),
prisma.task.count({ where: activeTasksFilter }),
prisma.task.count({
  where: {
    ...projectScope,
    status: TASK_STATUS.DONE,
  },
}),
prisma.task.count({
  where: {
    ...projectScope,
    status: TASK_STATUS.IN_PROGRESS,
  },
}),
```

`select: { id: true, name: true }` で2列だけ取っているのはこの後の表示に id と名前しか使わないからです。`select` を書かないと全列が返り、使わない説明文や日時まで通信に載ります。

完了件数の条件が `activeTasksFilter` ではなく `projectScope` になっている点に注目してください。取り消し済みのタスクは完了ではないので `NOT` の除外があっても件数は変わりません。母数の側だけが `activeTasksFilter` です。

件数の集計・後半と作業時間の合計:

```
// filepath: src/server/api/routers/report.ts(同じファイルの続き)
// 完成版: 件数の集計・後半と作業時間の合計

prisma.task.count({
  where: {
    ...projectScope,
    status: TASK_STATUS.IN_REVIEW,
  },
}),
prisma.task.count({
  where: {
    ...projectScope,
    status: TASK_STATUS.TODO,
  },
}),
prisma.task.aggregate({
  where: activeTasksFilter,
  _sum: { timeSpentMinutes: true },
}),
```

`aggregate` は合計値を返し、`count` は行数を返します。作業時間は1件ごとの分
数を足し合わせる必要があるので `_sum` を使います。`findMany` で全件を取って
から画面で足す書き方もできますがその場合はタスクの全行が通信に載ります。

Day 21: 統計カードを表示しよう

直近のタスクとステータス別の集計:

```
// filepath: src/server/api/routers/report.ts(同じファイルの続き)

// 完成版: 直近のタスクとステータス別の集計

prisma.task.findMany({
  where: activeTasksFilter,
  select: {
    id: true,
    title: true,
    status: true,
    priority: true,
  },
  orderBy: { updatedAt: 'desc' },
  take: 5,
}),

prisma.task.groupBy({
  by: ['status'],
  where: activeTasksFilter,
  _count: { _all: true },
}),
```

`take: 5` で5件に切っているのは直近の動きを見せるための一覧だからです。上限を付けないとタスクが1000件あるユーザーの画面で1000行が届きます。

`groupBy` はステータスの種類ごとに件数を数えて返します。ステータスごとに `count` を呼ぶ方法もありますが、種類が増えると呼び出しも増えます。ここではキャンセル済みを除くので、返るステータスは最大4種類です。`groupBy` ならデータベースにある種類の分だけ自動で並びます。

優先度別とプロジェクト別の集計:

```
// filepath: src/server/api/routers/report.ts(同じファイルの続き)
// 完成版: 優先度別とプロジェクト別の集計

prisma.task.groupBy({
  by: ['priority'],
  where: activeTasksFilter,
  _count: { _all: true },
}),

prisma.task.groupBy({
  by: ['projectId'],
  where: activeTasksFilter,
  _count: { _all: true },
  _sum: { timeSpentMinutes: true },
}),

prisma.task.groupBy({
  by: ['projectId'],
  where: {
    ...projectScope,
    status: TASK_STATUS.DONE,
  },
  _count: { _all: true },
}),

]);
```

Day 21: 統計カードを表示しよう

プロジェクト別の `groupBy` が2本ある理由は条件が違うからです。1本目は母数の件数と作業時間、2本目は完了だけの件数を数えます。1本にまとめる書き方もありますが条件の違う数を1回の集計で分けて出すにはもっと複雑な指定が必要になります。

`_count` と `_sum` を同じ集計で一緒に頼めるので1本目は件数と分数を1回で取っています。

割合と平均の計算:

```
// filepath: src/server/api/routers/report.ts(同じファイルの続き)
// 完成版: 割合と平均の計算

const totalTimeSpent =
totalTimeAggregate._sum.timeSpentMinutes ?? 0;

const averageTimePerTask = totalTasks > 0 ?
totalTimeSpent / totalTasks : 0;

const completionRate = totalTasks > 0 ? Math.round((
completedTasks / totalTasks) * 100) : 0;
```

3行すべてに `?? 0` か `totalTasks > 0` の確認が入っています。`_sum` は対象が1件も無いとき `null` を返し、割り算は分母が0のとき `NaN` になります。Reactは `null` をそのまま描画すると何も表示しません。`NaN` は数値表示の異常につながるため、どちらも0にそろえて返します。

`Math.round` で整数に丸めているのは完了率をパーセントの整数として見せるためです。丸める場所を画面側にすると他の画面で違う桁数になります。

プロジェクト別の集計の対応表:

```
// filepath: src/server/api/routers/report.ts(同じファイルの続き)
// 完成版: プロジェクト別の集計の対応表

const doneCountByProject = new Map(
  projectDoneGroups.map((group) => [group.projectId,
group._count._all]),
);

const taskStatsByProject = new Map(
  projectTaskGroups.map((group) => [
    group.projectId,
    {
      totalTasks: group._count._all,
      totalTimeSpent: group._sum.timeSpentMinutes ??
0,
    },
  ]),
);
```

`groupBy` が返すのは配列なのでそのままではプロジェクトごとの値を引けません。
`Map` に組み替えておくとその後の `projects.map()` の中で `id` を渡すだけで引
 けます。配列のまま `find` を呼ぶとプロジェクトの数だけ全体を探し直します。

戻り値の前半:

Day 21: 統計カードを表示しよう

```
// filepath: src/server/api/routers/report.ts(同じファイルの続き)
// 完成版: 戻り値の前半

return {
  totalProjects: projects.length,
  totalTasks,
  completedTasks,
  inProgressTasks,
  inReviewTasks,
  todoTasks,
  completionRate,
  totalTimeSpent,
  averageTimePerTask,
  recentTasks,
  statusData: statusGroups.map((group) => ({
    key: group.status,
    value: group._count._all,
  })),
  priorityData: priorityGroups.map((group) => ({
    key: group.priority,
    value: group._count._all,
  })),
}
```

`statusData` と `priorityData` を `key` と `value` という名前に置き換えているのは Day 22 でグラフに渡すときの形をそろえるためです。`status` と `priority` のまま返すとグラフの部品をステータス用と優先度用で 2 つ作るようになります。

項目の並びが空のときの戻り値と同じ順になっている点も確かめてください。並びを
そろえておくと項目の足し忘れを目で見つけられます。

Day 21: 統計カードを表示しよう

次のコードは `projectStats` を組み立てる処理です。

```
// filepath: src/server/api/routers/report.ts(同じファイルの続き)
// 完成版: projectStats の組み立て

projectStats: projects.map((project) => {
  const taskStats = taskStatsByProject.get(
project.id) ?? {
    totalTasks: 0,
    totalTimeSpent: 0,
  };

  const completedTaskCount = doneCountByProject.get(
project.id) ?? 0;

  return {
    id: project.id,
    name: project.name,
    totalTasks: taskStats.totalTasks,
    completedTasks: completedTaskCount,
    progress:
      taskStats.totalTasks > 0 ? (
completedTaskCount / taskStats.totalTasks) * 100 : 0,
    totalTimeHours: taskStats.totalTimeSpent / 60,
  };
}),
});
```

`?? { totalTasks: 0, totalTimeSpent: 0 }` が効くのはタスクが1件もないプロジェクトです。`groupBy` はタスクのある分だけ行を返すので空のプロジェクトは対応表に載りません。この既定値が無いとそのプロジェクトの行で `undefined` を読んで画面が落ちます。

`totalTimeHours` で60で割っているのはデータベースが分で持っている値を時間に直すためです。一方、カード用の `totalTimeSpent` と `averageTimePerTask` は分のまま返し、画面の計算式で時間へ換算します。返り値によって単位が異なるので、表の時間をさらに60で割らないようにしてください。

`src/server/api/root.ts`

ルーターの取り込み:

```
// filepath: src/server/api/root.ts
// 完成版: ルーターの取り込み
import { authRouter } from './routers/auth';
import { commentRouter } from './routers/comment';
import { projectRouter } from './routers/project';
import { reportRouter } from './routers/report';
import { searchRouter } from './routers/search';
import { taskRouter } from './routers/task';
import { createCallerFactory, createTRPCRouter } from
'./trpc';
```

今日足したのは `reportRouter` の1行だけです。並びがアルファベット順になっているのは Biome が整えるからで、追加した位置は気にしなくて大丈夫です。

手続きの一覧表:

Day 21: 統計カードを表示しよう

```
// filepath: src/server/api/root.ts(同じファイルの続き)
// 完成版: 手続きの一覧表
export const appRouter = createTRPCRouter({
  auth: authRouter,
  project: projectRouter,
  task: taskRouter,
  search: searchRouter,
  comment: commentRouter,
  report: reportRouter,
});

export type AppRouter = typeof appRouter;

export const createCaller = createCallerFactory(
  appRouter);
```

左に書いたキーがそのまま画面側の呼び名になります。`report: reportRouter`と書いたので `api.report.getOverview` で呼べます。ここを書き忘れるとルーターのファイルは正しく書けているのに画面側で `api.report` が見つからないというエラーが出ます。

こちらの並びは Day 18 で決めた時系列順で、アルファベット順ではありません。作った順に読めるほうが、どの機能がどの Day で入ったかを追えるからです。

`src/app/report/page.tsx`

画面の import:

```
// filepath: src/app/report/page.tsx
// 完成版: 画面の import
'use client';

import { AppLayout }
  from '@component/layout/app-layout';
import {
  Card, CardContent,
  CardHeader, CardTitle,
} from '@component/ui/card';
import { PageLoadingSpinner }
  from '@component/ui/loading-spinner';
import {
  Table, TableBody, TableCell,
  TableHead, TableHeader, TableRow,
} from '@component/ui/table';
import { api } from '@trpc/react';
```

1行目の `'use client'` が要ります。

`api.report.getOverview.useQuery()` はブラウザ側で動く仕組みなのでこの宣言が無いと、サーバーコンポーネントからこのフックを呼ぶためエラーになります。

`Table` から始まる6つの部品を一度に取り込んでいるのはテーブルの1行1列がそれぞれ別の部品になっているからです。`<table>` を直接書く方法でも表は作れますがその場合は枠線や余白の指定を自分で書くことになります。

データ取得と表示用の値:

Day 21: 統計カードを表示しよう

```
// filepath: src/app/report/page.tsx(同じファイルの続き)
// 完成版: データ取得と表示用の値
export default function ReportPage() {
  const { data: overview, isLoading } =
    api.report.getOverview.useQuery();

  const totalTasks = overview?.totalTasks ?? 0;
  const completionRate =
    overview?.completionRate ?? 0;
  const totalTimeHours =
    ((overview?.totalTimeSpent ?? 0) / 60)
      .toFixed(1);
  const averageTimeHours =
    ((overview?.averageTimePerTask ?? 0) / 60)
      .toFixed(1);

  if (isLoading) {
    return <PageLoadingSpinner />;
  }
}
```

`useQuery()` に引数が無いのはこの手続きが入力を取らないからです。誰の集計を返すかはサーバー側が Cookie から取り出したユーザーで決まります。画面から id を送る形にすると書き換えて他人の集計を見る道ができます。

4つの値を `return` の前で用意しているのは JSX の中に計算を書かないためです。JSX の中に

`((overview?.totalTimeSpent ?? 0) / 60).toFixed(1)` を直接書くとどこが表示でどこが計算か読み分けにくくなります。

`isLoading` の判定を値の準備より後に置いているのは `overview` が未定義のあいだも `?? 0` で受け止められるからです。判定を先に置いても動きますがこの並びだと値の定義が1か所にまとまります。

JSX — 見出し:

```
// filepath: src/app/report/page.tsx(同じファイルの続き)
// 完成版: JSX - 見出し

return (
  <AppLayout>
    <div className="space-y-6">
      <div>
        <h1 className="text-3xl
          font-bold tracking-tight">
          レポート・統計
        </h1>
        <p className=
          "text-muted-foreground">
          プロジェクトの進捗とタスクの状況を確認できます。
        </p>
      </div>
    </div>
  </AppLayout>
)
```

`<h1>` はページに1つだけ置きます。読み上げソフトや検索エンジンがそのページの主題として扱う見出しだからです。この後のカードやテーブルの見出しに `<h1>` を重ねると主題が複数あることになります。

`text-muted-foreground` は文字を薄い色にする指定です。見出しと説明文で濃さを変えるとどちらが主かが色だけで伝わります。

Day 21: 統計カードを表示しよう

JSX — 統計カードの前半2枚:

```
    { /* filepath: src/app/report/page.tsx(同じファイ  
ルの続き) */}  
    { /* 完成版: JSX — 統計カードの前半2枚 */}  
    <div className="grid grid-cols-1  
      sm:grid-cols-2 lg:grid-cols-4  
      gap-4">  
      <Card>  
        <CardContent className="pt-6">  
          <p className="text-sm  
            text-muted-foreground mb-1">  
            タスク数</p>  
          <p className="text-3xl font-bold">  
            {totalTasks}</p>  
        </CardContent>  
      </Card>  
      <Card>  
        <CardContent className="pt-6">  
          <p className="text-sm  
            text-muted-foreground mb-1">  
            完了率</p>  
          <p className="text-3xl font-bold">  
            {completionRate}%</p>  
        </CardContent>  
      </Card>
```

`grid-cols-1` から `lg:grid-cols-4` まで3段の指定があるので幅に応じて1列・2列・4列へ切り替わります。スマートフォンで4列にすると数字の桁が折り返して読めなくなります。

各カードで項目名を `text-sm` の薄い色、数字を `text-3xl font-bold` にしています。見せたいのは数字なので大きさと太さの差で目が先に数字へ行きます。両方を同じ大きさにすると4枚のカードがどれも同じ見た目になり、読者は数字を探すことになります。

Day 21: 統計カードを表示しよう

次のコードは統計カードの後半2枚です。

```
    { /* filepath: src/app/report/page.tsx(同じファイルの続き) */ }
    { /* 完成版: JSX - 統計カードの後半2枚 */ }
    <Card>
      <CardContent className="pt-6">
        <p className="text-sm text-muted-foreground mb-1">
          合計作業時間</p>
        <p className="text-3xl font-bold">
          {totalTimeHours}h</p>
      </CardContent>
    </Card>
    <Card>
      <CardContent className="pt-6">
        <p className="text-sm text-muted-foreground mb-1">
          平均作業時間/タスク</p>
        <p className="text-3xl font-bold">
          {averageTimeHours}h</p>
      </CardContent>
    </Card>
  </div>
```

数字の後ろに **h** を付けているのは単位が無いと 12 という数字を12時間とも12分とも読めるからです。完了率の側に **%** を付けているのも同じ考えです。

平均作業時間/タスク という項目名にしてあるのは何あたりの平均かを示すためです。「平均作業時間」だけでは1日あたりとも1人あたりとも読めます。

JSX — テーブルの見出し:

```

    { /* filepath: src/app/report/page.tsx(同じファイルの続き) */ }
    { /* 完成版: JSX — テーブルの見出し */ }
    <Card>
      <CardHeader>
        <CardTitle>
          プロジェクト統計</CardTitle>
        </CardHeader>
      <CardContent>
        <Table>
          <TableHeader>
            <TableRow>
              <TableHead className="w-[200px]">
                プロジェクト</TableHead>
              <TableHead className="text-right">
                タスク数</TableHead>
              <TableHead className="text-right">
                完了</TableHead>
              <TableHead className="text-right">
                進捗</TableHead>
              <TableHead className="text-right">
                作業時間</TableHead>
            </TableRow>
          </TableHeader>

```

Day 21: 統計カードを表示しよう

プロジェクト名の列だけ `w-[200px]` で幅を固定し、残りの4列を `text-right` で右へ寄せています。数字は桁の位置がそろっていると比べやすく、左寄せだと桁数の違いで数の大小が読み取りにくくなります。

名前の列に幅を決めておくとプロジェクト名の長さで表の形が変わりません。長い名前が1つ入るだけで数字の列が押しつぶされる状態を防げます。

JSX — テーブルの行・前半:

```
    { /* filepath: src/app/report/page.tsx(同じ  
ファイルの続き) */}  
    { /* 完成版: JSX — テーブルの行・前半 */}  
    <TableBody>  
      {overview?.projectStats.map((stat) => (  
        <TableRow key={stat.id}>  
          <TableCell  
            className="font-medium">  
              {stat.name}</TableCell>  
          <TableCell  
            className="text-right">  
              {stat.totalTasks}</TableCell>  
          <TableCell  
            className="text-right">  
              {stat.completedTasks}</TableCell>
```

`overview?.` の `?.` がデータが届く前の状態を受け止めています。`isLoading` の判定を通っているので通常はここへ `undefined` が来ませんが通信が失敗した場合は `overview` が `undefined` のまま描画されます。この1文字が無いとその場で画面が落ちます。

`key={stat.id}` は React が行を見分けるための目印です。プロジェクトの id を渡しているので並び順が変わっても行の中身が入れ替わりません。

Day 21: 統計カードを表示しよう

次のコードはテーブルの行の後半と閉じタグです。

```
        { /* filepath:
src/app/report/page.tsx(同じファイルの続き) */
        { /* 完成版: JSX - テーブルの行・後半と
閉じタグ */

        <TableCell
            className="text-right">
            {stat.progress.toFixed(
1)}%</TableCell>

        <TableCell
            className="text-right">
            {stat.totalTimeHours.toFixed(1)}h
        </TableCell>
        </TableRow>
        )})
    </TableBody>
    </Table>
    </CardContent>
    </Card>
  </div>
</AppLayout>
);
}
```

`toFixed(1)` で小数第1位までに丸めています。`progress` はサーバー側で $(完了 / 全体) * 100$ を計算しただけの値なので丸めないと `33.33333333333333%` のような表示になります。カードの `completionRate` はサーバーで整数に丸めますが、表の `progress` は画面で小数第1位まで表示します。この教材では表示先ごとに精度が異なるため、丸める場所も分かれています。

閉じタグは `</TableBody>`、`</Table>`、`</CardContent>`、`</Card>`、`</div>`、`</AppLayout>` の順で、開いた順の逆になっています。1つでも抜けるとブラウザに赤いエラー画面が出て足りない場所が書かれています。

`src/component/layout/app-layout.tsx`

Day 08 で作った長いファイルなので今日触った2か所だけを載せます。それ以外の行はDay 20までに書いた状態を残してください。

アイコンの取り込み:

```
// filepath: src/component/layout/app-layout.tsx
// 完成版: アイコンの取り込み
import {
  BarChart,
  ClipboardList,
  FolderOpen,
  LayoutDashboard,
  ListTodo,
  Logout,
  Search,
} from 'lucide-react';
```

Day 21: 統計カードを表示しよう

今日足したのは **BarChart** の1語です。他の6つは Day 08 から Day 20 までに並べたもので消すとサイドバーのアイコンがその項目だけ出なくなります。名前をアルファベット順に並べてあるのは Biome の並べ替えに合わせるためで、順番が違っていると保存した瞬間に並べ替えられ、触っていない行まで差分に入ります。

サイドバーのメニュー項目:

```
// filepath: src/component/layout/app-layout.tsx
// 完成版: Day 20 までに並べたメニュー項目
const menuItems: MenuItem[] = [
  {
    text: 'ダッシュボード',
    icon: <LayoutDashboard className="h-5 w-5" />,
    path: '/dashboard',
  },
  {
    text: 'プロジェクト',
    icon: <FolderOpen className="h-5 w-5" />,
    path: '/project',
  },
  {
    text: 'マイタスク',
    icon: <ListTodo className="h-5 w-5" />,
    path: '/my-task',
  },
  {
    text: 'タスク',
    icon: <ClipboardList className="h-5 w-5" />,
    path: '/task',
  },
]
```

Day 21: 統計カードを表示しよう

ここまでの4件は Day 08・Day 09・Day 13・Day 17 で足したものです。配列を途中で切って載せているのでこの時点では閉じカッコがありません。続きは次のブロックです。

```
// filepath: src/component/layout/app-layout.tsx(同じ配列の続き)
// 完成版: 検索と、今日足したレポート

{
  text: '検索',
  icon: <Search className="h-5 w-5" />,
  path: '/search',
},
{
  text: 'レポート',
  icon: <BarChart className="h-5 w-5" />,
  path: '/report',
},
];
```

末尾の「レポート」1件が今日の追加です。前の5件は Day 08・Day 09・Day 13・Day 17・Day 20 で足したものがそのまま残ります。並べた順がそのまま画面の上から下の順になるので途中へ差し込むとメニューの並びが教材の画面と変わります。`path` の `/report` は `src/app/report/page.tsx` の置き場所と対応していて綴りがずれるとメニューは出るのに押した先が404になります。

完成形の参考コード: 完成版には `src/app/report/page.tsx` と `src/server/api/routers/report.ts` があります。ただし今日書いたコードとは一部が異なります。違いは4つです。1つ目は完成版の画面に円グラ

フが2枚並んでいる点です。これは Day 22 で足します。2つ目は完成版の画面に週次レポートへのリンクがある点です。これは Day 23 で足します。3つ目は完成版の `report.ts` に `getOverview` 以外の手続きも入っていて `root.ts` には Day 24 で追加する `user` も登録されている点です。4つ目は完成版の画面が通信エラーと本当の0件を分けて表示する点です。これは Day 26 で足します。この4か所は違って当たり前だと思って読んでください。(販売用 ZIP に完成版の `src/` は入っていません。ここに挙げた違いは完成版がどう書かれているかの説明として読んでください)。

今日のまとめ

- ☐ `api.report.getOverview` の役割を理解しました
- ☐ server 集計済みデータをカードに表示できました
- ☐ 4枚の統計カードを表示できました
- ☐ プロジェクト統計テーブルを表示できました
- ☐ レスポンシブグリッドを適用できました

つまずきポイント

エラー / 問題	原因	解決方法
NaN が表示される	<code>overview</code> が未取得のまま参照している	<code>?? 0</code> でフォールバック
時間が分で表示	60 で割り忘れ	<code>/ 60</code> で時間に変換
カードが縦並び	グリッドクラス不足	sm/lg ブレークポイント

今日学んだ用語

用語	意味
getOverview	レポート用の集計済みデータを返す API
groupBy	サーバー側で件数を分類して集計する処理
toFixed(1)	小数点以下1桁に丸める
aggregate	サーバー側で合計値をまとめて計算する処理
early return	条件付きで先に表示を返す手法

理解チェック

今日書いたコードを見ながら答えてみてください。答えは各問のすぐ下にあります。

Q1. `activeTasksFilter` は `projectScope` に何を足した条件ですか。

A. `NOT: { status: TASK_STATUS.CANCELLED }`、つまり中止したタスクを除く条件を足したものです。`projectScope` の側は「自分が参加中で、しかもアーカイブされていないプロジェクトのタスク」までを決めています。2段に分けてあるので中止を数えたい集計では `projectScope` のほうを使えます。

Q2. 完了率の分母を `activeTasksFilter` の件数から `projectScope` の件数に変えると数字はどう動きますか。

A. 中止したタスクが分母に入るので完了率は下がります。中止したタスクが `DONE` になることはなく、分子は変わらないまま分母だけが増えるためです。10 件のうち 5 件を終えて 3 件を中止したプロジェクトなら母数 7 件なら 71%、母数 10 件なら 50%になります。

Q3. 参加しているプロジェクトが 0 件のときに `null` や `{}` ではなく 13 項目すべてを `0` と `[]` で埋めて返すのはなぜですか。

A. API の戻り値の形を、データがある場合とそろえるためです。現在の画面は `overview?.totalTasks ?? 0` と書いているので、`null` でもカードは 0 を表示します。ただし、データが 0 件なのか戻り値が欠けているのかを区別できません。`{}` では `projectStats` などが `undefined` になり、`.map()` を呼ぶ箇所でエラーになります。数値を 0、配列を `[]` で返せば、画面側は 0 件を通常の集計結果として扱えます。

追加課題: 未対応の件数をカードにする

理解チェック Q3 の戻り値を使い、未対応タスクの件数を表示しましょう。サーバーが返している値なら新しい集計 API を作らずに使えます。

前提は今日の `/report` が表示できることです。

`src/app/report/page.tsx` の統計カードを参考に、「未対応」のカードを 1 枚追加してください。`getOverview` の戻り値一覧から該当する項目を選び、値がまだ無いときは 0 を表示します。既存の 4 枚は残します。

表示中の未対応の件数を控えます。自分が管理するプロジェクトに未対応の「課題 21」タスクを 1 件作ります。レポートを再読み込みし、追加前より未対応の数字が 1 増えれば成功です。

Day 21: 統計カードを表示しよう

数字が増えない場合はそのプロジェクトがアーカイブされていないか確認してください。カードの参照先も戻り値一覧と比べます。確認後は追加したカードを外し、課題用タスクを削除します。画面側でタスク全件を数え直す必要がない理由も説明してみましょう。

次回予告

Day 22 ではレポートページにグラフを追加します。Recharts で円グラフを表示し、タスクの分布を可視化します。

次に読むもの

- 前の日: Day 20^{*1}
- 次の日: Day 22^{*2}
- 全体の地図: 学びのロードマップ^{*3}
- 目次: カリキュラム目次^{*4}
- 詰まったとき: トラブルシューティング^{*5}
- 言葉の意味: 用語集^{*6}

-
1. <https://drive.google.com/file/d/1ribgG0JH4lD0DAkaYd5VragjRzxn8wyX/view?usp=drivesdk>
 2. https://drive.google.com/file/d/1kA8jhIcg7LJ3EGTWygNnABUihixSCD_C/view?usp=drivesdk
 3. https://drive.google.com/file/d/10py_p-6ry4RDlxBSyzPn4rtzZe74ND-T/view?usp=drivesdk
 4. <https://drive.google.com/file/d/19g0cKLajEWi7s9LeWF804DB9j5kK1dt4/view?usp=drivesdk>
 5. <https://drive.google.com/file/d/1OyjFWiGvZBm9QKcnKiougPZidjsn7kqR/view?usp=drivesdk>
 6. <https://drive.google.com/file/d/1RtElBpzkvXsoTpj4CTKPko8jnQDcPsh/view?usp=drivesdk>

奥付

タイトル	Day 21: 統計カードを表示しよう
著者	磯貝光佑
版	第1版(2026年9月19日)

© 2026 磯貝光佑

本書は購入者個人の利用に限ります。本書に掲載されたコードの写経、および自分のプロジェクトへの転用・改変は自由です。本書(PDF・Markdown)および付属コードの再配布・転売・第三者との共有は、形態を問わず禁止します。